SOCIAL MEDIA ANALYTICS · ITMO UNIVERSITY

Lab 1 — Kairos Dataset

Did this post beat what its author usually gets?

Yaroslava Galimova · Social Media Analytics, Agents & LLMs · ITMO University · autmn, 2026



Final metrics — test set, 17,623 posts

METRIC	VALUE
macro-F1	**0.6236**
QWK	**0.5438**
Accuracy	**0.6650**
F1 below	**0.5071**
F1 typical	**0.7298**
F1 above	**0.6339**

Baseline to beat: 0.5410 macro-F1 → improvement +15.3%.

How to run (from kairos-dataset/):

1 Unpack the core archive into data/
2 pip install -r requirements.txt
3 python runlab.py — ~40–55 min on CPU
4 python evaluate.py --predictions results/predictions/lab1.csv

Predictions land in results/predictions/lab1.csv (17,623 rows).

TASK

The task

Did this post beat what its own author usually gets?

Classes — cutoffs fixed on train, applied unchanged to test:

Below

< ×0.577 of the author's usual reactions

Typical

between

Above

> ×3.182

Data (Kairos release, SOMIN 2026):

71,395

labelled train posts / 3,295 authors

17,623

test posts / 824 authors

Split by author — every author sits entirely on one side

Metric: macro-F1 over the three ordered classes.

Key rule: the author's "usual level" for a post uses only that author's strictly earlier posts — no leakage, no peeking.

FEATURES

Features

Total dense: 61 numeric features + 256 text SVD dims.

01

Content structure (19)

Text classification

Characters / words / lines / paragraphs, avg word length, hashtags, mentions, URLs, ⟨ORG_n⟩ / ⟨PERSON_n⟩ placeholders, emoji, capitalised words, has ? / ! / digits / money symbols, posting hour and weekday.

02

Author history (14)

LinkedIn: six sources

Median, mean, std, min, max, IQR of earlier log(1+reactions); count of earlier posts; median and std over the last 3 posts; median vs level; last residual; mean residual over 3 and 10 posts; days since previous post.

03

Relative to the author's usual (10)

For words, hashtags, mentions, URLs, emoji: post – author's median. Plus the five history medians used as the baseline for those differences.

04

Text (TF-IDF → SVD 256)

tf.idf + SVD

Word 1–2-grams (50k) + char_wb 3–5-grams (50k), concatenated, reduced with TruncatedSVD to 256 dims — otherwise LightGBM training on 100k sparse features takes 20–40 min with no quality gain.

05

Media / audience (7)

multimodal

Counts of images, videos, PDFs, has_media, encoded content_type, log_followers, age_days. Plus 7 pixel-level stats from up to 3 images per post (width, height, aspect, file size, brightness, saturation, distinct colours).

06

BM25 to past posts (4)

BM25 retrieval

For each post: BM25 similarity to the author's earlier posts, plus the mean residual and mean log(1+reactions) of the 5 most similar past posts.

NOTES

Three of the six feature groups come straight from Lecture 2 -- text classification, the LinkedIn six-source profile, and late-fusion multimodality. The BM25 group comes from Lecture 3's retrieval section: the idea that “similar past posts of this author are the strongest signal of how this one will do”. Every feature that uses the author's history is computed with shift(1).expanding(min_periods=5), so a post never enters its own level

MODELLING

Models & fusion

Three experts, late-fused — late fusion.

EXPERT	INPUT	MODEL
Dense	61 numeric features	LightGBM × 3 seeds (42, 7, 123)
Text	TF-IDF-SVD 256	LightGBM × 3 seeds (42, 67, 58)
Dense	61 numeric features	CatBoost

CLASS SPLIT

20 / 60 / 20

Below / Typical / Above

Blend: p = 0.40·p_dense + 0.30·p_text + 0.30·p_cat

Class imbalance handled with weights

class_weight="balanced" on LightGBM, auto_class_weights="Balanced" on CatBoost. The classes are 20 / 60 / 20, and macro-F1 punishes collapsing onto “typical”.

Post-processing: calibration on the validation split

argmax on probabilities is not the macro-F1 optimum. Two class weights (w_below, 1.0, w_above) are grid-searched on the validation split to maximise macro-F1, then frozen and applied unchanged to test. Final weights: [0.786, 1.0, 0.786].

Two ideas — late fusion and choosing the right metric — carried this design. Separate experts per modality, blended by weighted probability. Then calibration to make sure the blend is actually optimising macro-F1, not just argmax accuracy. The calibration step alone was worth about +0.01 on test, but its more important effect is making sure extremes are not over-predicted.

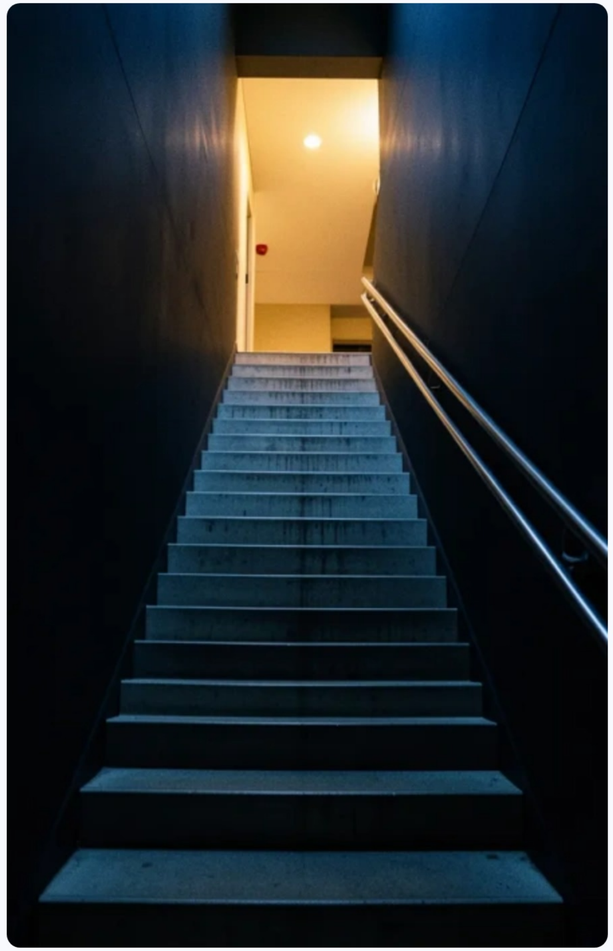
EXPERIMENTS

What I tried, what worked, what didn't

STEP	CHANGE	EFFECT ON MACRO-F1
0	Baseline (published)	**0.5410**
1	Dense only (content + history + relative)	**~0.55–0.58 (val)**
2	+ TF-IDF + SVD 256	**+0.03–0.04 (val)**
3	+ CatBoost in the blend	**+0.01 (val)**
4	+ Media pixel features (PIL)	**+0.001 (test: 0.6196 → 0.6207)**
5	+ BM25 to past posts + 3-seed ensembles	**+0.003 (test: 0.6207 → 0.6236)**
6	SVD 512 instead of 256	**+0.005 (val), ~2× training time → reverted to 256**

Take-away: the biggest single gain was combining author history with text (steps 1 → 2). Media content is the biggest remaining opportunity that this submission does not exploit. I also tried CLIP image embeddings, but local encoding would have taken more than 14 hours, and given the deadline I chose not to wait and focused on other features instead.

The dense-plus-text combination did most of the heavy lifting. Media pixel features, BM25 and ensembles together added less than 1 point, but they are cheap and stable.



ERRORS

Error analysis

Confusion matrix (rows = truth, columns = prediction)

	pred below	pred typical	pred above	RECALL
truth below (3,541)	1,879	1,620	42	**0.531**
truth typical (10,491)	1,871	7,753	867	**0.739**
truth above (3,591)	120	1,383	2,088	**0.581**

Where the errors come from (5,903 total misclassifications):

Extremes → typical: 3,003 (51%) — boundary posts (×0.6, ×3.0) are genuinely ambiguous.

Typical → extremes: 2,738 (46%) — the model over-fires on ambiguous middle cases.

Below ↔ above: 162 (3%) — the true ordinal mistake. Rare, which is good for QWK.

What this means for each class

above

precise (0.697) but conservative (0.581). Better to miss an above than to fabricate one.

below

worst precision (0.486). The model over-calls below — the class is noisier and harder to pin down.

typical

dominant (0.730 F1) but it absorbs errors from both sides.



Extremes → typical · 51%

Typical → extremes · 46%

Below ↔ above · 3%

Next lever for errors: longer history windows, explicit trend features (rising vs falling author), and media content (not just pixel statistics).

WHAT'S NEXT

Limitations & next steps

What this submission does not do

— Media content is unused. Pixel features are weak.

— Text is TF-IDF-SVD. Sentence embeddings (all-MiniLM-L6-v2) would likely add 0.3–0.5 macro-F1 and share the same download limitation.

— Pseudo-labelling for test authors (self-training from confident predictions)

What I would try next

1 CatBoost with text features enabled (uses raw text directly, no TF-IDF needed).

2 Sentence embeddings as a fourth expert in the blend.

3 Optuna over LightGBM depth / learning rate / leaves — small but reliable.

Reproducibility: everything is deterministic given SEED = 42. Fresh clone → pip install -r requirements.txt → python runlab.py → same 0.6236 on the same machine class. The full pipeline (loading, image features, BM25, validation, final training) runs in ~40–55 minutes on CPU.

